

Hierarchical Actor-Orchestrated State Management with DIRAM-Backed Epistemic Validation

OBINexus Computing - Aegis Framework Division
 Technical Specification for Actor Sub-ConOps Architecture
 Document Classification: Production Infrastructure
 Compliance: NASA-STD-8739.8, AEGIS-PROOF-1.2

Abstract—We present the hierarchical state resolution model for Actor-orchestrated systems, extending the OBIAI Actor class through sub-conceptual task decomposition with DIRAM-backed memory governance. Each EA Actor autonomously manages task lifecycles using a TO-DO $\rightarrow$ DOING $\rightarrow$ DONE progression model, maintaining epistemic validation at 95.4% confidence threshold. The system implements strategic rollback cascades when success:failure ratios fall below 1:2, ensuring self-correcting behavior through cryptographically traced state transitions. This architecture represents deployed production infrastructure, not theoretical design, providing forensic-level accountability through SHA-256 receipt logs and verb-noun conceptual modeling aligned with the Actor class tuple $\alpha = (S, \mathcal{C}, \Phi, \Psi, \epsilon)$.

I. INTRODUCTION

The hierarchical state resolution model extends the Actor class defined in the OBIAI framework through systematic sub-conceptual decomposition. Building upon the categorical foundation where Actors navigate infinite-dimensional semantic manifolds, we implement a production-ready state management system that maintains epistemic discipline while enabling autonomous task orchestration.

Definition 1 (Actor Class Extension). Given an Actor $\alpha = (S, \mathcal{C}, \Phi, \Psi, \epsilon)$ where $\epsilon \geq 0.954$, the hierarchical state extension introduces:

- Sub-conceptual decomposition function $D : S \rightarrow 2^S$
- State lifecycle automaton $L : S \times \mathcal{C} \rightarrow S$
- DIRAM trace function $T : S \rightarrow \{0, 1\}^{256}$

This extension enables Actors to decompose high-level missions into epistemically validated sub-tasks while maintaining the dimensional innovation property essential to the Actor paradigm.

II. DIRAM HARDWARE FAULT-TOLERANT ARCHITECTURE

A. Core State Structure

The hierarchical state management system anchors to DIRAM's cryptographic memory governance through the following C structure:

```
typedef struct {
    uint64_t state_id;
    char parent_state_hash[65]; // SHA-256
    trace
    verb_noun_concept_t intent;
    float result_metric;
    float proof_confidence; // >=
    0.954
    state_flag_t status_flag; //
    Lifecycle position
    uint8_t error_count;
    uint64_t timestamp;
    diram_state_allocation_t* diram_trace;
} hierarchical_state_t;

typedef enum {
    STATE_TODO = 0x01,
    STATE_DOING = 0x02,
    STATE_DONE = 0x04,
    STATE_BLOCKED = 0x08,
    STATE_ROLLEDBACK = 0x10
} state_flag_t;
```

Listing 1. DIRAM-backed hierarchical state structure

B. Memory Allocation with Trace Linking

Every state allocation generates a cryptographic receipt ensuring forensic traceability:

```
diram_state_allocation_t*
diram_allocate_state_memory(
    hierarchical_state_t* state,
    const char* intent_tag
) {
    // Enforce epistemic constraint
    if (state->proof_confidence <
        EPISTEMIC_THRESHOLD) {
        return NULL;
```

```

8     }
9
10    // Generate SHA-256 receipt
11    diram_allocation_t* base =
12        diram_alloc_traced(
13            sizeof(hierarchical_state_t),
14            intent_tag);
15
16    // Link to blockchain for audit trail
17    gitraf_blockchain_append_state(
18        state->state_id,
19        state->parent_state_hash);
20
21    return create_state_allocation(base, state);
22 }

```

Listing 2. DIRAM state allocation implementation

III. TASK LIFECYCLE MANAGEMENT WITH WATERFALL GATES

A. State Transition Automaton

The lifecycle progression follows a deterministic automaton with epistemic validation at each gate:

Theorem 1 (Lifecycle Soundness). For any state $s \in S$ with confidence $c_s \geq 0.954$, the transition function L guarantees that $L(s, C) = s'$ implies that the verify-trace- Φ operation validates the transition ($s \rightarrow s'$) as TRUE.

Proof. Each transition invokes the audit-transition- Φ function which validates the epistemic signature Φ before permitting state advancement. The DIRAM trace function T generates cryptographic proof of transition validity. $\square$

B. Waterfall Gate Implementation

```

1  int enforce_waterfall_gate(
2      hierarchical_state_t* state,
3      waterfall_gate_t gate
4  ) {
5      switch (gate) {
6          case GATE_1_TODO_VALIDATION:
7              if (state->proof_confidence <
8                  0.954) {
9                  state->status_flag =
10                     STATE_BLOCKED;
11                     emit_trace("GATE_1_FAILED",
12                                state->state_id);
13                     return -1;
14             }
15             break;
16
17         case GATE_2_DOING_PROGRESS:
18             float ratio =
19                 calculate_success_failure_ratio(
20                     state);
21             if (ratio < 0.5) { // Below 1:2
22                 threshold

```

```

17         initiate_cascade_rollback(
18             state);
19         return -1;
20     }
21     break;
22
23     case GATE_3_DONE_VERIFICATION:
24         emit_verification_proof(state);
25         commit_state_to_diram(state);
26         break;
27     }
28     return 0;
29 }

```

Listing 3. Waterfall gate enforcement

IV. ROLLBACK CASCADE PROTOCOL

A. Strategic Rollback Mechanism

When trial-and-error patterns emerge ($\text{error_count} \geq 2$), the system initiates the emit-rollback- Φ operation:

Algorithm 1 Cascade Rollback Protocol

- 1: **Input:** Failed state s_f with confidence $c_f < 0.954$
- 2: **Output:** Rollback cascade receipt R
- 3: $D \leftarrow \text{trace-dependency}(s_f)$ {Using trace-dependency- Φ }
- 4: $\text{depth} \leftarrow \min(|D|, 5)$ {Limit cascade depth}
- 5: **for** $d = 0$ **to** depth **do**
- 6: $S_d \leftarrow \{s \in D : \text{depth}(s) = d\}$
- 7: **for each** $s \in S_d$ **do**
- 8: $s.\text{confidence} \leftarrow s.\text{confidence} \times (1 - 0.1d)$
- 9: $s.\text{status} \leftarrow \text{STATE_TODO}$
- 10: memoize-delta(s, c_f) {Using memoize-delta- Φ }
- 11: generate-receipt(s) {Using generate-receipt- Φ }
- 12: **end for**
- 13: **end for**
- 14: **return** append-trace(R) {Using append-trace- Φ }

B. Success:Failure Ratio Enforcement

The system maintains epistemic discipline through continuous ratio monitoring:

```

def assess_state_continuation(self, state):
    """Implements trial-and-improvement with
    rollback"""
    # Check trial-and-error lock
    if state.confidence < 0.954 and state.
        error_count >= 2:
        return self._initiate_rollback(state)

    # Check success:failure ratio
    ratio = self._calculate_success_ratio(
        state)

```

```

9   if ratio < self.rollback_cascade_threshold
10      : # < 0.5
11      return self.
12         _strategic_rollback_cascade(state)
13
14      # Normal progression
15      if state.status_flag == StateFlag.DONE:
16          return self._emit_verification_proof(
17              state)
18
19      else:
20          return self._update_state(state)

```

Listing 4. Python implementation of ratio enforcement

V. ACTOR SUB-CONOPS INTEGRATION

A. Alignment with Actor Class Tuple

The hierarchical state model preserves the Actor’s dimensional innovation property while adding structured task management:

Proposition 1 (Innovation Preservation). For Actor $\alpha = (S, \mathcal{C}, \Phi, \Psi, \epsilon)$ with hierarchical extension, the dimensional innovation property holds:

$$\exists \tau : S \rightarrow S \text{ where } \tau \notin \text{span}(\mathcal{C}) \implies \exists s \in S : D(\tau(S)) \ni s$$

This ensures that Actor-driven innovations translate to actionable sub-tasks while maintaining epistemic boundaries.

B. Verb-Noun Conceptual Modeling

Each state intent follows the formalized triplet structure (V, N, Φ) :

```

1  typedef struct {
2      char verb[32];           // Action operation
3      char noun[32];          // Domain object
4      float phi_vector[8];    // Epistemic
5                               signature
6  } verb_noun_concept_t;
7
8  // Example instantiation
9  verb_noun_concept_t intent = {
10     .verb = "predict",
11     .noun = "failure",
12     .phi_vector = {0.97, 0.95, 0.98, 0.96,
13                   0.94, 0.99, 0.95, 0.97}
14 };

```

Listing 5. Verb-noun concept implementation

VI. TURING SOUNDNESS IN TASK DECOMPOSITION

Theorem 2 (Decomposition Completeness). The hierarchical state system with DIRAM backing achieves Turing-complete task orchestration while maintaining epistemic soundness.

Proof. We construct a correspondence between state transitions and Turing machine computation:

- 1) States in S encode Turing configurations
- 2) Lifecycle transitions simulate state machine evolution
- 3) DIRAM provides unbounded memory through linked allocations
- 4) Rollback mechanism implements rejection states
- 5) The validate-confidence- Φ operation ensures only sound computations proceed

The 95.4% threshold prevents non-deterministic branching while cascade protocols enable recovery from computational dead-ends. $\square$

VII. COMPLIANCE AND AUDIT FRAMEWORK

A. AEGIS-PROOF Traceability

Every state transition generates auditable proof through:

- commit-state- Φ : Persistence with cryptographic receipt
- anchor-hardware- Φ : Physical memory binding for forensics
- compute-ratio- Φ : Continuous success metric validation

B. NASA-STD-8739.8 Adherence

The system satisfies safety-critical requirements through:

- 1) **Deterministic Execution**: State transitions follow formal automaton
- 2) **Bounded Resources**: DIRAM enforces $\epsilon(x) \leq 0.6$ constraint
- 3) **Graceful Degradation**: Cascade rollback prevents catastrophic failure
- 4) **Formal Verification**: All paths traceable through SHA-256 receipts

VIII. PRODUCTION DEPLOYMENT ARCHITECTURE

```

class ActorSubConOpsOrchestrator:
    """Production-ready hierarchical task
    orchestration"""

    def __init__(self):
        self.epistemic_threshold = 0.954
        self.rollback_cascade_threshold = 0.5
        self.diram = DIRAMInterface()

    def process_mission(self, actor, mission):
        # Decompose using dimensional
        # innovation
        states = self.decompose_mission(actor,
                                         mission)

```

```

12
13     # Process each state through lifecycle
14     for state in states:
15         while state.status_flag !=
16             STATE_DONE:
17             transition = self.
18                 process_state_lifecycle(
19                     state)
20
21             if transition ==
22                 StateTransition.ROLLBACK:
23                 self.
24                     handle_cascade_recovery
25                     (state)
26
27             elif transition ==
28                 StateTransition.BLOCKED:
29                 self.resolve_dependencies(
30                     state)
31
32     return self.compile_mission_proof(
33         states)

```

Listing 6. Complete orchestrator implementation

IX. CONCLUSION

The hierarchical Actor-orchestrated state management system represents deployed infrastructure achieving self-correcting AI orchestration through:

- DIRAM-backed memory governance with cryptographic traceability
- 95.4% epistemic validation threshold enforcement
- Strategic rollback cascades maintaining 1:2 success:failure ratios
- Verb-noun conceptual modeling for semantic task representation
- Waterfall gate compliance for systematic validation

This architecture operates continuously across OBINexus deployments, transforming Actor-level dimensional innovations into tractable, verifiable sub-tasks while maintaining the mathematical rigor demanded by safety-critical AI systems.

VERB-NOUN CONCEPT GLOSSARY

- anchor-~~Bind~~ epistemic state to physical memory substrate. 3
- append-~~Attach~~ state transition to immutable DIRAM log. 2
- audit-~~Inspect~~ state lifecycle compliance with confidence metrics. 2
- commit-~~Finalize~~ state persistence to DIRAM with receipt generation. 3
- compute-~~Calculate~~ success:failure metrics for cascade detection. 3

- emit-rollback-~~Rollback~~ rollback event with epistemic signature for state recovery. 2
- generate-~~Produce~~ SHA-256 trace for forensic accountability. 2
- memoize-~~Store~~ confidence degradation for future reference. 2
- trace-dependency-~~Map~~ hierarchical state relationships for rollback scope. 2
- validate-~~Assess~~ confidence against 95.4% threshold. 3
- verify-~~Validate~~ cryptographic integrity of state transition history with epistemic signature Φ . 2